

Colour-Based Object Segmentation using the HSV Model

Detecting and Extracting Coloured Objects in MATLAB

1. Aim

To detect and separate objects of a chosen colour from an image in MATLAB by converting the image to the **HSV colour model** and applying a simple **colour threshold** to create a mask for each colour.

2. Introduction

Many machine-vision tasks need to find objects by their **colour** — for example, picking red apples, reading a traffic light, or sorting coloured items on a conveyor belt. This is called **colour-based segmentation**.

Doing this directly in the usual **RGB** image is difficult, because in RGB the colour and the brightness are mixed together in all three channels. The **HSV** colour model solves this by keeping the colour (Hue) separate from the brightness, which makes it very easy to select one colour. This is a basic topic of **colour image processing** in the Machine Vision syllabus.

Colour-based detection is widely used in **automation and robotics** (colour sorting, object picking) and in agriculture and quality inspection, so this work links to **SDG 9 (Industry, Innovation and Infrastructure)**.

3. Theory

3.1 RGB and HSV Colour Models

In the **RGB** model every pixel is a mix of Red, Green and Blue. The problem is that if the light gets brighter or darker, all three values change, so the same colour has many different RGB values.

The **HSV** model describes a colour the way people do, using three separate numbers:

- **Hue (H)** — the actual colour (red, green, blue, yellow ...).
- **Saturation (S)** — how pure or strong the colour is (grey → vivid).
- **Value (V)** — how bright it is (dark → light).

Because the colour is held in a single channel (Hue), selecting "all the red pixels" becomes very simple and much less affected by lighting.

3.2 Colour-Based Segmentation

To find a colour, we keep only the pixels whose **Hue lies in the range of that colour**. We also require a minimum **Saturation** and **Value** so that very pale or very dark pixels (which have no clear colour) are ignored. The result is a **binary mask**: white (1) where the colour is present and black (0) everywhere else.

3.3 Building the Mask

Each colour is a band of Hue values. In MATLAB the Hue runs from 0 to 1, so for example green is roughly 0.25–0.45 and blue is 0.55–0.72. **Red is a special case** because it sits at

both ends of the scale (near 0 and near 1), so its mask uses two ranges joined together (Hue < 0.05 OR Hue > 0.95).

3.4 Using the Mask

Once the mask is made it can be used to **extract** the coloured objects (keep only the masked pixels and blank out the rest) or to **measure** them — counting the mask pixels gives the number of pixels of that colour and its percentage of the image area, which can be used to detect or count coloured objects.

4. Methodology

Implemented in MATLAB using the Image Processing Toolbox function `rgb2hsv`. The steps are:

1. Read the colour (RGB) image.
2. Convert it to the HSV colour model with `rgb2hsv` and split the H, S and V channels.
3. For each colour, keep the pixels whose Hue is in that colour's range (with a minimum S and V) to make a binary mask.
4. Display the mask for each colour and extract the objects of one chosen colour.
5. Count the mask pixels to get the number and percentage area of each colour.

Data flow: RGB image → convert to HSV → threshold Hue (+ S, V) → colour mask → extract objects / count pixels.

5. MATLAB Code

After converting to HSV, each colour mask is one line that checks the Hue range (plus a minimum saturation and value). Keep **objects.png** in the same folder as the script.

```
%% Colour-Based Object Segmentation using the HSV Model

clc; clear; close all;

% STEP 1 - Read the colour image
I = imread('objects.png');

% STEP 2 - Convert to HSV and split the channels
hsv = rgb2hsv(I);
Hc = hsv(:,:,1);  Sc = hsv(:,:,2);  Vc = hsv(:,:,3);

% STEP 3 - Make a mask for each colour (Hue is 0..1 in MATLAB)
% Red wraps around 0 and 1, so it uses two ranges.
redMask    = ((Hc < 0.05) | (Hc > 0.95)) & Sc > 0.35 & Vc > 0.25;
greenMask   = (Hc >= 0.25 & Hc <= 0.45) & Sc > 0.35 & Vc > 0.25;
blueMask    = (Hc >= 0.55 & Hc <= 0.72) & Sc > 0.35 & Vc > 0.25;
yellowMask  = (Hc >= 0.12 & Hc <= 0.20) & Sc > 0.35 & Vc > 0.25;

% STEP 4 - Show the masks
figure;
subplot(2,2,1); imshow(redMask);    title('Red');
subplot(2,2,2); imshow(greenMask); title('Green');
subplot(2,2,3); imshow(blueMask);  title('Blue');
subplot(2,2,4); imshow(yellowMask); title('Yellow');

% Extract the red objects (blank out everything that is not red)
redObjects = I;
redObjects(repmat(~redMask, [1 1 3])) = 255;
figure; imshow(redObjects); title('Extracted Red Objects');

% STEP 5 - Count pixels and percentage area of each colour
total = numel(Hc);
```

```
fprintf('Red    : %6d px  (%.2f%%)\n', nnz(redMask), 100*nnz(redMask)/total);
fprintf('Green  : %6d px  (%.2f%%)\n', nnz(greenMask), 100*nnz(greenMask)/total);
fprintf('Blue   : %6d px  (%.2f%%)\n', nnz(blueMask), 100*nnz(blueMask)/total);
fprintf('Yellow : %6d px  (%.2f%%)\n', nnz(yellowMask), 100*nnz(yellowMask)/total);
```

6. The Image in the HSV Model

Figure 1 shows the input image and its three HSV channels. In the **Hue** channel each colour becomes a different gray level, in the **Saturation** channel the coloured objects are bright (pure colour) while the white background is dark, and the **Value** channel shows the brightness. This separation is what makes the colour easy to pick.

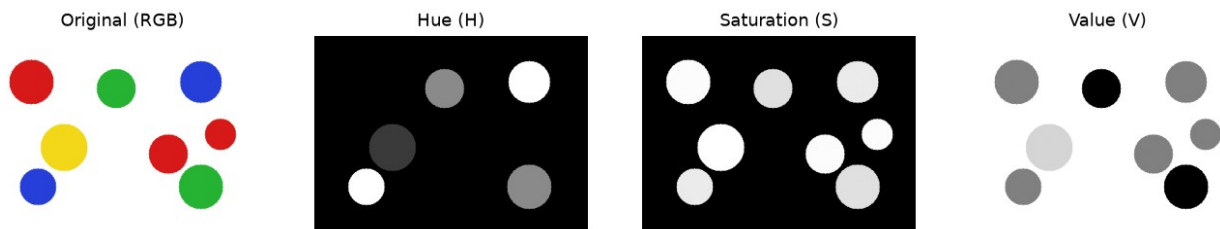


Figure 1: The original RGB image and its Hue, Saturation and Value channels.

7. Results and Output Images

Figure 2 shows the binary mask produced for each colour — each mask correctly contains only the objects of that colour (three red, two green, two blue and one yellow). Figure 3 shows the red objects extracted from the image using the red mask, with everything else blanked out.

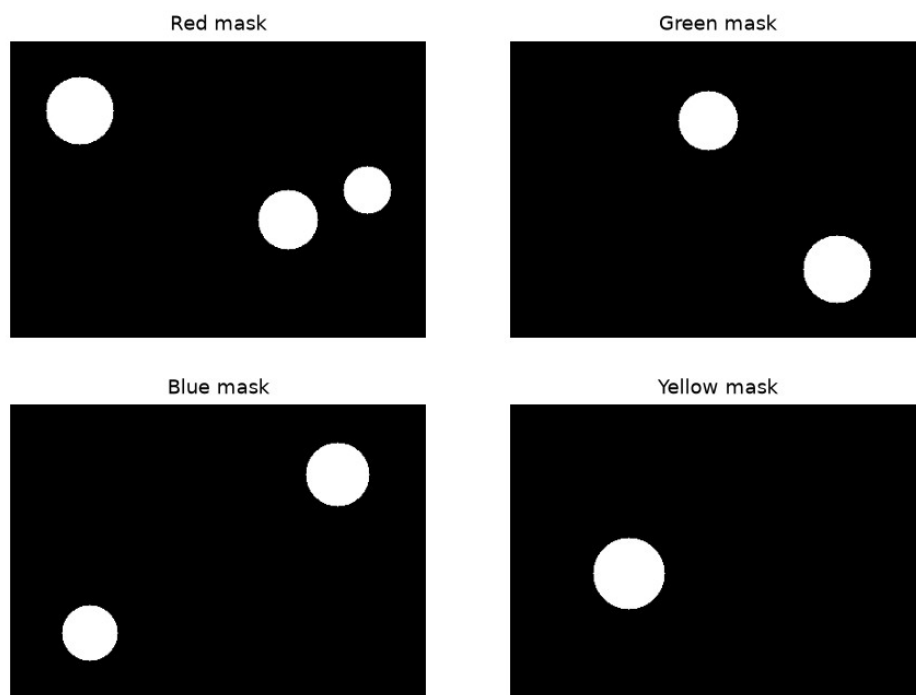


Figure 2: Binary masks obtained for the red, green, blue and yellow colours.

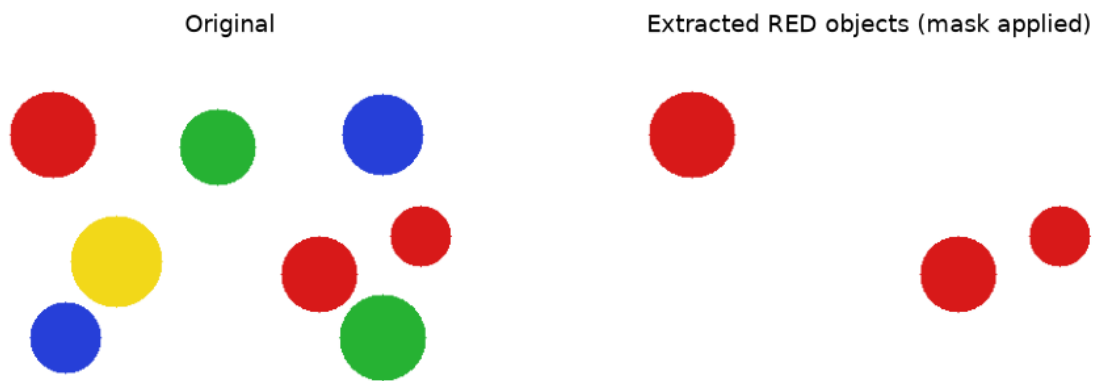


Figure 3: The red objects extracted from the image using the red mask.

8. Quantitative Results

The table lists the Hue range used for each colour and the number and percentage of pixels found (image size $300 \times 420 = 126\,000$ pixels).

Colour	Hue Range (0-1)	Pixels Found	% of Image
Red	> 0.95 or < 0.05	8 239	6.54
Green	0.25 – 0.45	6 446	5.12
Blue	0.55 – 0.72	5 662	4.49
Yellow	0.12 – 0.20	4 053	3.22

9. Result Analysis

- Each colour was correctly separated using only its **Hue range** plus a minimum saturation and value — this shows how well HSV isolates colour from brightness.
- The **pixel counts match the objects** in the image (red is highest because there are three red objects; yellow is lowest because there is only one), so the method can also be used to **count or measure** coloured objects.
- Adding the **saturation and value limits** was important — it removed the white background and any dull pixels, so only clearly coloured pixels were kept.

The method worked cleanly here because the objects had strong, distinct colours. Its main limitation is that **colours that look similar in hue** (for example orange and red) can overlap, and the hue ranges may need small adjustments for real photographs with mixed lighting.

10. Advantages and Limitations

Aspect	Details
Advantages	Very simple and fast; HSV separates colour from brightness so it works under different lighting; easy to detect several colours at once; gives count and area.

Aspect	Details
Limitations	Hue ranges must be set for each colour; similar colours can overlap; very dark, white or pale areas have no reliable hue; real photos may need tuning.

11. Applications

Colour-based segmentation is used for **colour sorting** of products and recyclables, **fruit grading and ripeness detection** in agriculture, **traffic-light and road-sign detection** in vehicles, **object picking by colour** in robotics, skin detection, and colour-based quality inspection on production lines.

12. Conclusion

Colour-based object segmentation was implemented in MATLAB by converting the image to the **HSV colour model** and thresholding the **Hue** (with a minimum saturation and value) to build a mask for each colour. All four colours — red, green, blue and yellow — were separated correctly, and the pixel counts matched the number of objects of each colour. HSV makes colour segmentation **simple and reliable** because it keeps colour separate from brightness. The project meets CO-1 and CO-2 and supports SDG 9 (Industry, Innovation and Infrastructure).

13. Course Outcome and SDG Mapping

CO / SDG	How this project addresses it
CO-1	Implements a colour-image-processing algorithm (HSV segmentation) in MATLAB.
CO-2	Applies image processing to a practical field — colour sorting / object detection.
SDG 9	Supports industry and automation — colour detection is used in sorting, robotics and inspection.

14. References

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Addison-Wesley.
- [2] A. R. Smith, "Color Gamut Transform Pairs (HSV model)," *ACM SIGGRAPH*, 1978.
- [3] Scott E. Umbaugh, *Computer Vision and Image Processing*, Prentice-Hall.
- [4] MATLAB Documentation — Image Processing Toolbox: `rgb2hsv`, `imshow`.